



ARIENNE ALVES NAVARRO
CLARISSE LACERDA PIMENTEL
MARIA RITA RIBEIRO RESENDE
THALES RODRIGUES RESENDE

RELATÓRIO - ENTREGA 2
ANALISADOR SINTÁTICO (BISON)

1. GRAMÁTICA COMPLETA EM BNF

```
<programa> ::= <lista_comandos>

<lista_comandos> ::= <comando>
                    | <lista_comandos> <comando>

<comando> ::= <declaracao> SEMICOLON
            | <tipo> ID LPAREN <parametros> RPAREN <bloco>
            | <tipo> ID LPAREN RPAREN <bloco>
            | <atribuicao> SEMICOLON
            | <comando_if>
            | <comando_while>
            | <bloco>
            | <io_stmt>
            | <chamada_func> SEMICOLON
            | RETURN <expressao> SEMICOLON

<bloco> ::= LBRACE <lista_comandos> RBRACE
        | LBRACE RBRACE

<comando_if> ::= IF LPAREN <expressao> RPAREN <comando>
              | IF LPAREN <expressao> RPAREN <comando> ELSE <comando>

<comando_while> ::= WHILE LPAREN <expressao> RPAREN <comando>

<tipo> ::= INT
        | FLOAT

<declaracao> ::= <tipo> <lista_decl_itens>

<lista_decl_itens> ::= <decl_item>
                    | <lista_decl_itens> COMMA <decl_item>

<decl_item> ::= ID
              | ID ASSIGN <expressao>

<atribuicao> ::= ID ASSIGN <expressao>

<expressao> ::= <expressao> PLUS <expressao>
              | <expressao> MINUS <expressao>
              | <expressao> MULT <expressao>
              | <expressao> DIV <expressao>
              | <expressao> MOD <expressao>
              | <expressao> AND <expressao>
              | <expressao> OR <expressao>
              | <expressao> EQ <expressao>
              | <expressao> NE <expressao>
              | <expressao> LT <expressao>
              | <expressao> LE <expressao>
              | <expressao> GT <expressao>
              | <expressao> GE <expressao>
              | NOT <expressao>
              | MINUS <expressao>
              | LPAREN <expressao> RPAREN
              | NUM_INT
              | NUM_DEC
              | ID
```

```
| <chamada_func>

<parametros> ::= <parametro>
               | <parametros> COMMA <parametro>

<parametro> ::= <tipo> ID

<chamada_func> ::= ID LPAREN <argumentos> RPAREN
                | ID LPAREN RPAREN

<argumentos> ::= <expressao>
                | <argumentos> COMMA <expressao>

<io_stmt> ::= PRINT LPAREN <expressao> RPAREN SEMICOLON
            | READ LPAREN ID RPAREN SEMICOLON
```

2. DECISÕES DO PROJETO

Durante a concepção e implementação da Etapa 2 (Análise Sintática), o grupo adotou algumas premissas arquiteturais e metodológicas para garantir a robustez do compilador:

2.1. Resolução de Ambiguidade de Fluxo (Dangling Else)

Para solucionar, o parser foi estruturado para associar automaticamente o *else* ao *if* mais próximo, seguindo a convenção adotada pela maioria das linguagens imperativas. Isso evita ambiguidades sintáticas na interpretação de condicionais aninhadas.

2.2. Recuperação de Erros (Panic Mode Recovery)

Foi adotada uma estratégia de recuperação sintática baseada no *token* sincronizador ;. Quando um erro é encontrado dentro de um comando, o *parser* descarta *tokens* até encontrar um ponto e vírgula, permitindo que o compilador reporte múltiplos erros sintáticos durante uma única compilação, sem interromper imediatamente o processo de análise.

2.3. Estratégia de Reporte de Erros Sintáticos

O *parser* reporta a posição do *token* em que a inconsistência sintática é detectada, utilizando as informações de linha e coluna fornecidas pelo analisador léxico. Essa abordagem permite localizar o ponto em que a análise sintática deixa de corresponder à gramática definida, o que pode fazer o erro ser indicado algumas posições após sua origem real.

2.4. Consolidação da Gramática de Expressões

Em vez de fragmentar expressões matemáticas e lógicas em múltiplos não-terminais recursivos (como termo e fator), optou-se por centralizar todas as operações no não-terminal *<expressao>*. A precedência e a associatividade dos operadores (como a prioridade da multiplicação sobre a adição e a associatividade do unário negativo) foram especificadas no cabeçalho do Bison por meio das declarações *%left*, *%right* e *%prec*. Essa abordagem simplificou a estrutura da gramática e reduziu a complexidade de manutenção do *parser*.

2.5. Organização da linguagem em comandos e blocos

A linguagem foi estruturada em torno da abstração de comando, permitindo que estruturas condicionais e de repetição aceitem tanto instruções isoladas quanto blocos delimitados por chaves (*{}*), aumentando a flexibilidade sintática e aproximando o comportamento da linguagem ao encontrado em linguagens imperativas tradicionais.

2.6. Otimização da Tabela de Símbolos (Hashmap)

Para otimizar o desempenho do analisador léxico, a Tabela de Símbolos foi refatorada de um vetor estático de busca linear para uma tabela de dispersão (*hashmap*). A implementação utiliza uma função de espalhamento com multiplicador primo e resolve colisões por meio de encadeamento separado (listas encadeadas com alocação dinâmica). Essa mudança reduz a complexidade temporal de busca e inserção de $O(N)$ para uma média de $O(1)$, garantindo alta eficiência na compilação.

2.7. Metodologia de Desenvolvimento e Controle de Versão

Para o gerenciamento do código-fonte, utilizou-se o Git integrado ao GitHub. Essa abordagem garantiu a sincronização contínua do repositório, permitindo o desenvolvimento colaborativo assíncrono e mitigando conflitos de integração no arquivo central do analisador sintático.

3. DIFICULDADES ENCONTRADAS

A principal dificuldade durante a modelagem da gramática livre de contexto residuiu no tratamento de ambiguidades inerentes à sintaxe de linguagens *C-like*. Durante as compilações iniciais, o Bison reportou conflitos de *Shift/Reduce* na geração do autômato LR(0), que precisaram ser tratados pontualmente:

3.1. Conflito do Dangling Else

Durante o desenvolvimento da gramática, o Bison identificou um conflito *Shift/Reduce* clássico relacionado ao problema do *dangling else*, observado no arquivo *parser.output* do autômato LR(0). Esse conflito ocorria porque o analisador não conseguia decidir se deveria reduzir um comando *if* incompleto ou deslocar (*shift*) o *token* ELSE para empilhá-lo. Para resolver o problema, foram utilizadas regras de precedência fictícias com `%nonassoc LOWER_THAN_ELSE` e `%nonassoc ELSE`. Ao atribuir a precedência mais baixa à regra do *if* simples e uma precedência maior à regra do *if/else*, forçamos o autômato a associar corretamente o *else* ao *if* mais próximo.

3.2. Conflito do Menos Unário (Unary Minus)

Outra dificuldade encontrada foi o sobreamento do operador de subtração (MINUS) com o operador de inversão de sinal (menos unário). Como ambos utilizam o mesmo símbolo léxico (-), o Bison gerou um conflito ao tentar definir a precedência da operação. A solução adotada pelo grupo foi a criação de uma diretiva de precedência máxima no topo do arquivo (`%right UMINUS`) e a sua aplicação direta na regra de expressões unárias utilizando o comando explícito `%prec UMINUS`, garantindo que a inversão de sinal seja resolvida antes de qualquer outra operação aritmética.

4. CONJUNTOS FIRST E FOLLOW

Como o Bison utiliza um autômato *bottom-up* LALR(1), ele suporta nativamente regras com recursão à esquerda. Os conflitos de *Shift/Reduce* gerados por essa ambiguidade são resolvidos diretamente pelas diretivas de precedência e associatividade (`%left`, `%right` e `%prec`).

Isso dispensa transformações teóricas como estratificação ou eliminação de recursão. Assim, a análise foca apenas no não-terminal raiz *Expressao*, cuja produção generalizada é:

$$\text{Expressao} \rightarrow \text{Expressao OP Expressao} \mid (\text{Expressao}) \mid \text{OP}_{\text{unario}} \text{Expressao} \mid \text{ID} \mid \text{NUM}$$

Onde:

- OP: Conjunto de todos os operadores binários suportados (PLUS, MINUS, MULT, DIV, MOD, AND, OR, EQ, NE, LT, LE, GT, GE).
- $\text{OP}_{\text{unario}}$: Operadores que afetam um único operando à direita (NOT e o menos unário MINUS).

- ID e NUM: Identificadores de variáveis e literais numéricos base (NUM_INT, NUM_DEC).
- (Expressao): Representa o aninhamento por parênteses (LPAREN e RPAREN).

As tabelas a seguir detalham o mapeamento estrutural dos conjuntos FIRST e FOLLOW obtidos:

Não-terminal	FIRST
Expressao	{ LPAREN, NOT, MINUS, ID, NUM_INT, NUM_DEC }

Não-terminal	FOLLOW
Expressao	{ \$, RPAREN, SEMICOLON, COMMA, PLUS, MINUS, MULT, DIV, MOD, AND, OR, EQ, NE, LT, LE, GT, GE }

5. ESTADOS DO AUTÔMATO COM CONFLITOS

Após análise do arquivo gerado pelo *parser*, não foram observados conflitos *shift/reduce* ou *reduce/reduce* na gramática final. O caso clássico do *dangling else* foi resolvido corretamente pelo *parser*, associando o *else* ao *if* mais próximo, sem gerar conflito na versão final.

6. ARQUIVO DE TESTE E SUA SAÍDA

Para validar a robustez do analisador sintático e a eficácia das regras de recuperação de erro, elaborou-se um arquivo de código mesclando estruturas válidas da linguagem (declarações de escopo, desvios de fluxo, operações lógicas e matemáticas) com erros léxicos e sintáticos intencionais.

- **Arquivo de Entrada**

```
/*
comentário inútil
*/

int a;
int b, c;
int flag;
int ativo, concluido;

int z

a = 5;
b = a;
c = a + b * 3;
flag = 1;
ativo = 0;
concluido = ativo;

int w;;

a = 3 + 2 * 5;
b = (a + 1) * 4 - 2;
c = -a + (b - -c) / 2;
```

```
a = ;
```

```
flag = a < b;
```

```
flag = a <= b;
```

```
flag = a > b;
```

```
flag = a >= b;
```

```
flag = a == b;
```

```
flag = a != b;
```

```
if (a < b
```

```
    c = 1;
```

```
flag = 1 && 0;
```

```
flag = (a < b) || (c == a);
```

```
flag = !flag;
```

```
flag = !(a >= b && c != a);
```

```
if (a >= 10 && a <= 20 || !(c == 5))
```

```
    a = a + 1;
```

```
while ()
```

```
    a = a + 1;
```

```
@
```

```
int S = -2;
```

```
P = -0;
```

```
if (a < b)
```

```
    a = b;
```

```
if (a == b)
```

```
    c = 1;
```

```
else
```

```
    c = 2;
```

```
a = 5 +* 2;
```

```
//dangling else
```

```
if (a < b)
```

```
    if (b < c)
```

```
        a = b;
```

```
    else
```

```
        c = a;
```

```
while (a < 10)
```

```
    a = a + 1;
```

```
while (a < 20) {
```

```
    a = a + 1;
```

```
    print(a);
```

```
}
```

```
print(a);
```

```
read(b);
```

```
{
```

```

int x, y;
x = 10;
y = 20;
print(x);
print(y);
}

else
    a = 1;

```

- Saída

ERRO SINTATICO: syntax error na Linha 12, Coluna 1
ERRO SINTATICO: syntax error na Linha 19, Coluna 7
ERRO SINTATICO: syntax error na Linha 25, Coluna 5
ERRO SINTATICO: syntax error na Linha 35, Coluna 5
ERRO SINTATICO: syntax error na Linha 45, Coluna 8

ERRO LEXICO: '@' na Linha 48, Coluna 1
ERRO SINTATICO: syntax error na Linha 61, Coluna 8
ERRO SINTATICO: syntax error na Linha 89, Coluna 1

Analise sintatica concluida.

=== TABELA DE SIMBOLOS ===

Identificador	Linha	Coluna

a	5	5
b	6	5
c	6	8
flag	7	5
ativo	8	5
concluido	8	12
z	10	5
w	19	5
S	50	5
P	51	1
x	82	9